State Machines – Part 3
K. Sridharan

Review

- We have discussed a few examples of analysis of sequential circuits using state table and state diagram.
- We will now discuss two important concepts in state machines: the Mealy and the Moore machine models.
- We will then touch upon the notion of state reduction.

The Mealy Model and the Moore model

- The most general model of a sequential circuit has inputs, outputs and internal states.
- It is common to distinguish between two models of sequential circuits, namely the Mealy model and the Moore model. They differ only in the way the output is generated.

mealy → present & input
moore → present

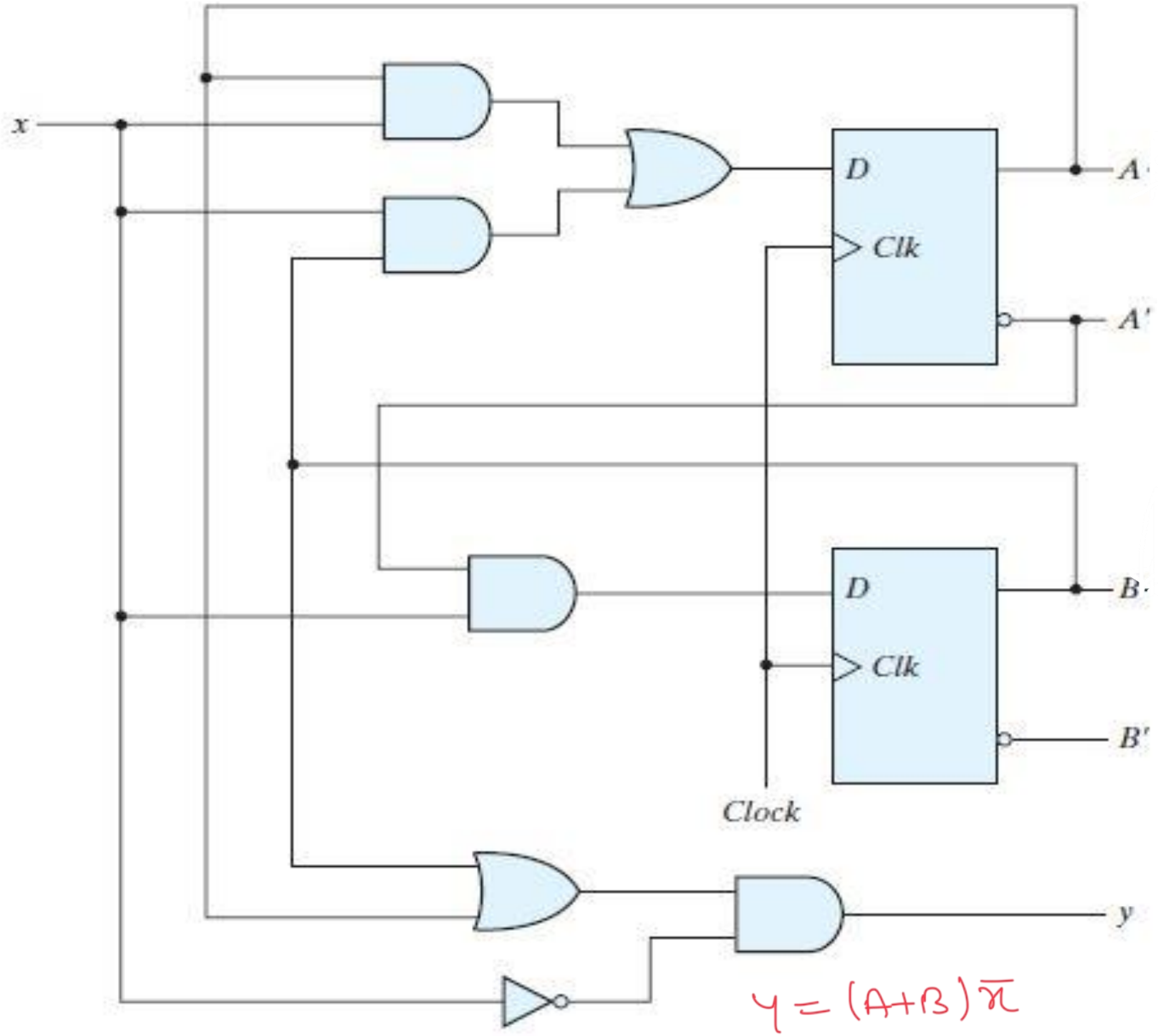
More on the Mealy Model and the Moore model

- In the Mealy model, the output is a function of both the present state and the input.
- In the Moore model, the output is a function of only the present state.
- The term Finite State Machine (FSM) is also used while referring to these. The Mealy model is called as a Mealy FSM or Mealy machine.
- The Moore model is also called as a Moore FSM or Moore machine.

Can go back to circuits shown

- We have discussed a few examples of analysis of sequential circuits where we have seen Mealy and Moore models.
- The first circuit we saw (sample sequential circuit in part 1) is an example of Mealy model (or Mealy machine)
- The picture in Example 3 is an example of the Moore machine.
- We depict these again next.

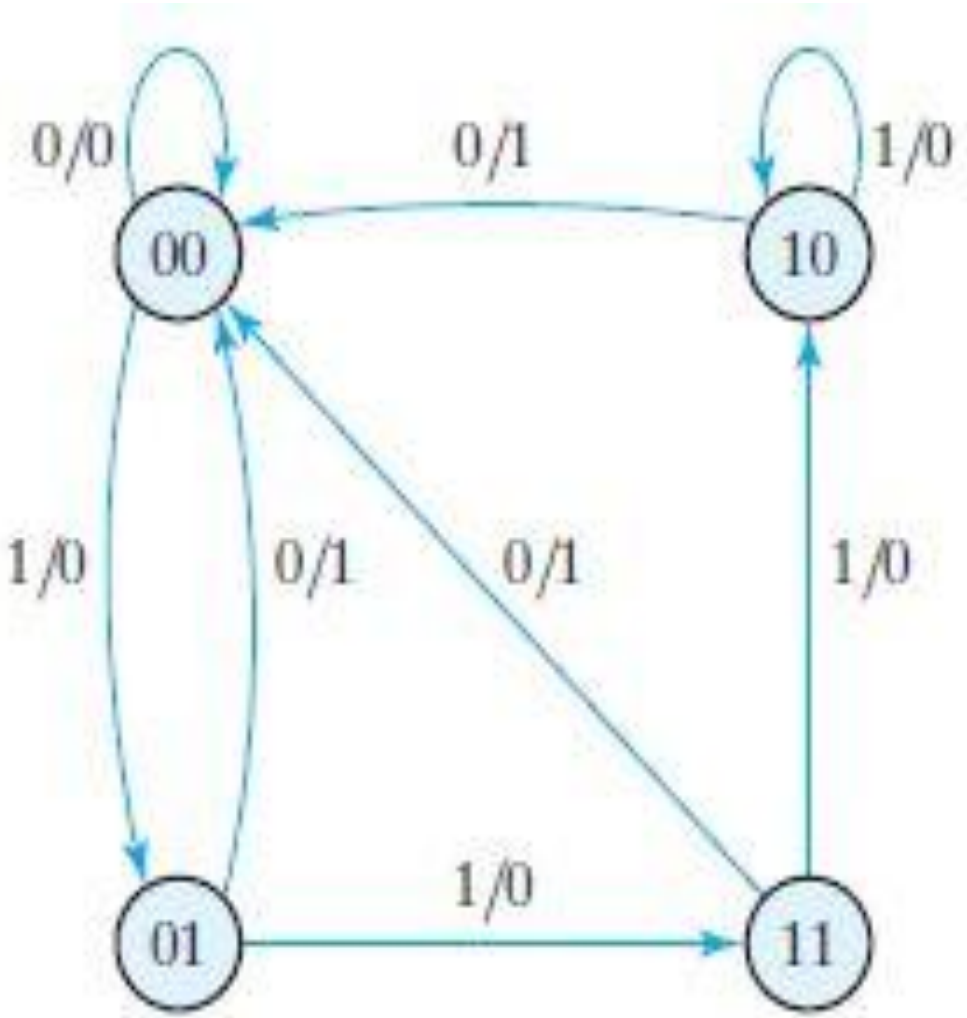
Example of Mealy machine



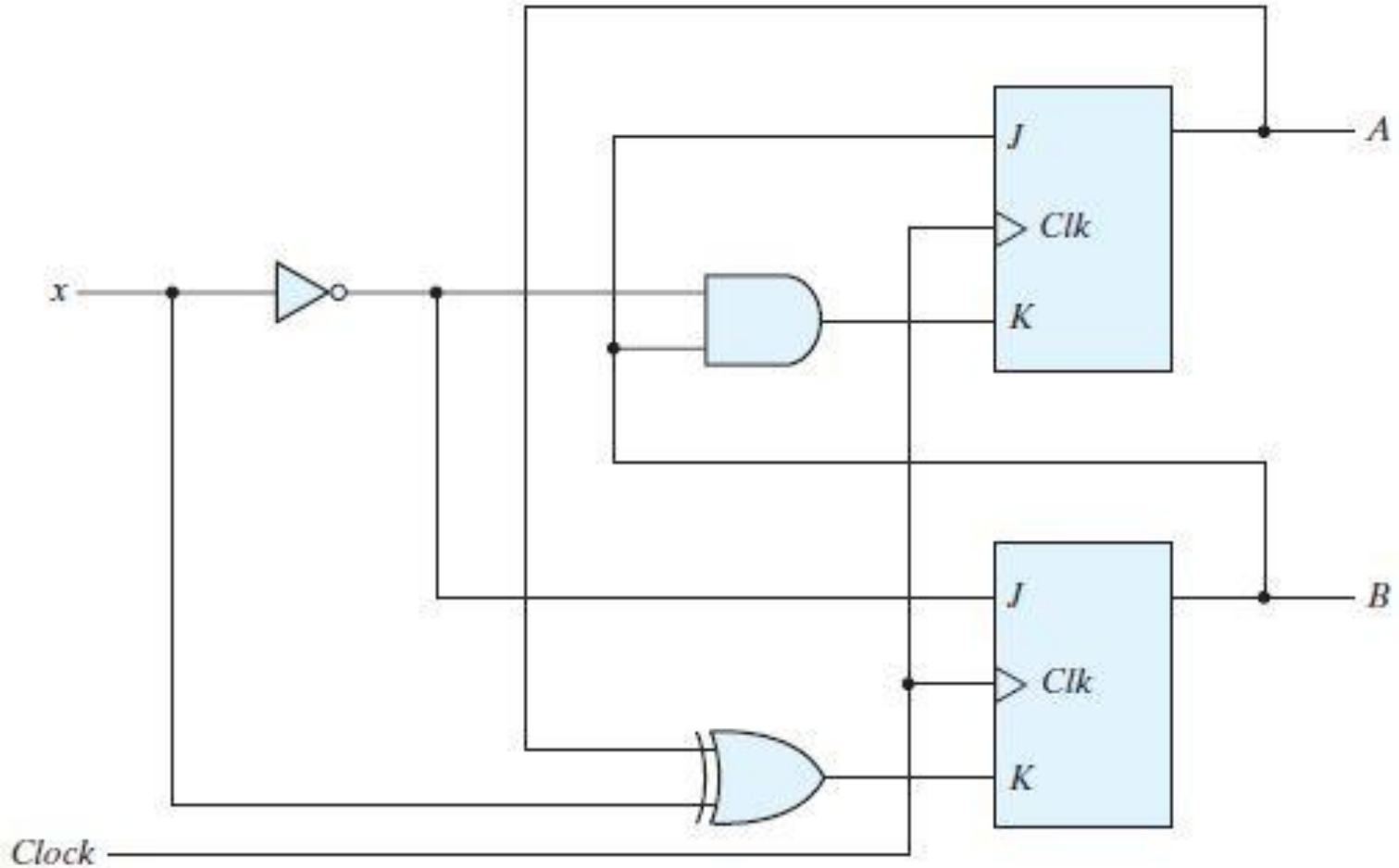
Explanation

- Output y is a function of both input x and the present state of A and B.
- You may also recall that the corresponding state diagram (discussed earlier) shows both the input and output values separated by a slash along the (directed) arcs between the nodes. It is depicted again next.

State Diagram



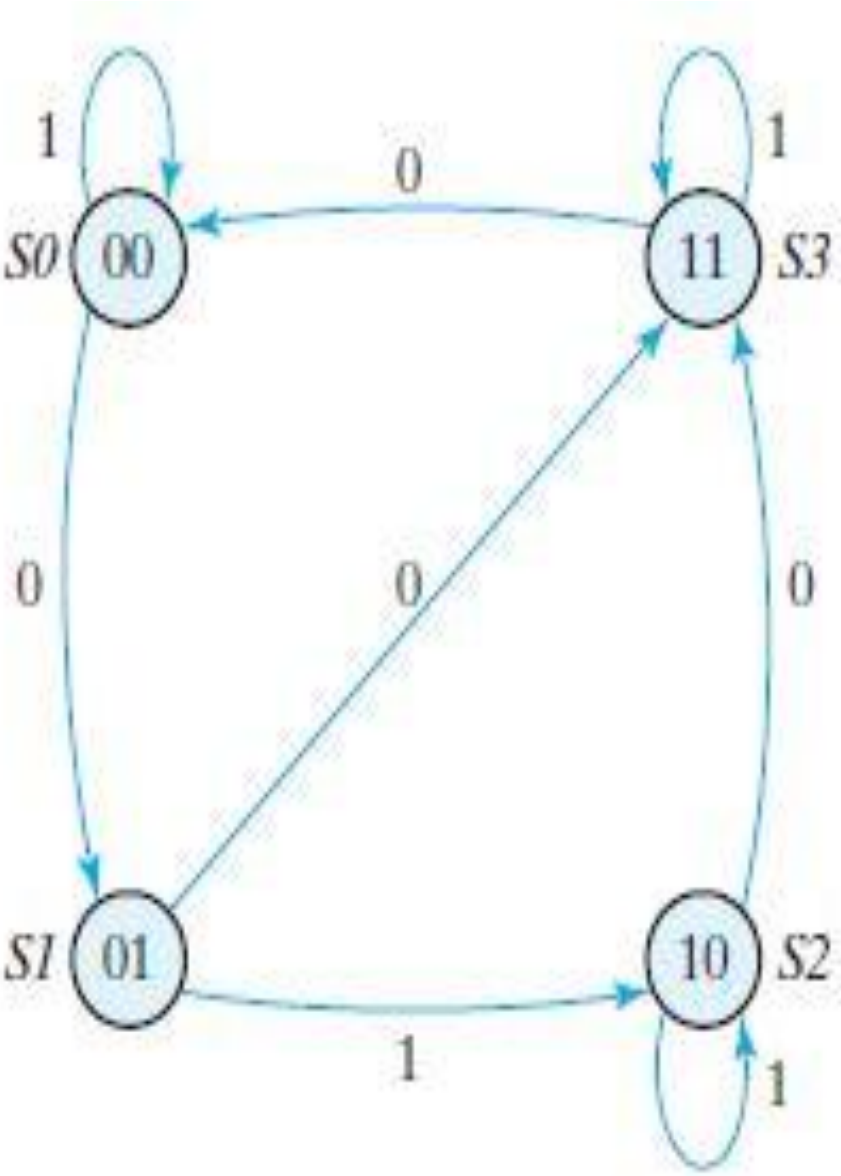
Example of Moore model



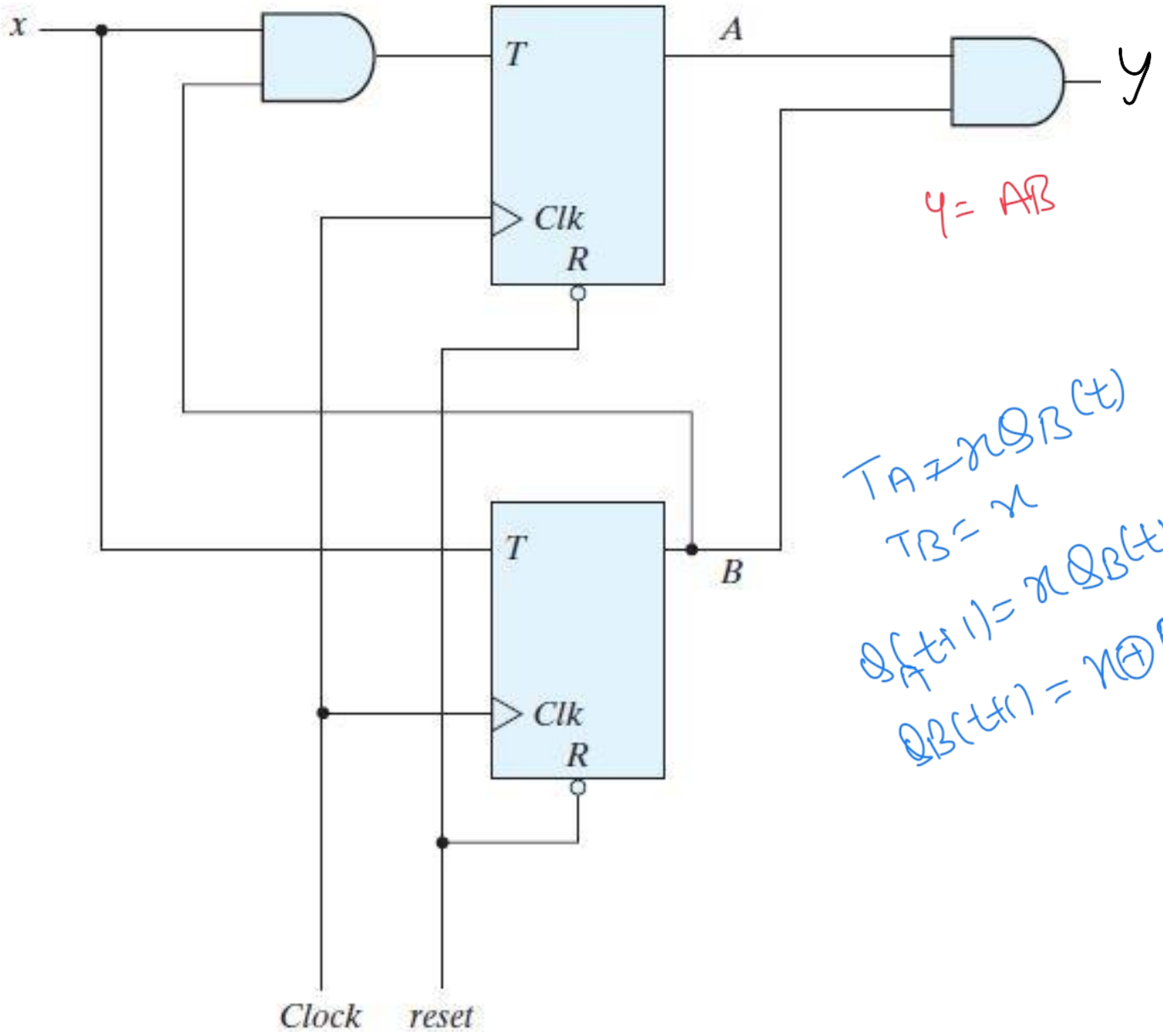
Explanation

- In the Moore model circuit shown, note that the output is a function of the present state only.
- The outputs here refer to the flip-flop states marked inside circles in the state diagram (see next slide).

State Diagram



Now consider this diagram



$y = AB$

$T_A = x \oplus B(t)$

$T_B = x$

$Q_A(t+1) = x \oplus B(t) \oplus Q_A(t)$

$Q_B(t+1) = x \oplus Q_B(t)$

Which FSM (Mealy/Moore) does this diagram represent ?

- Question for students

Identify the FSM (Mealy/Moore)

- Question for students

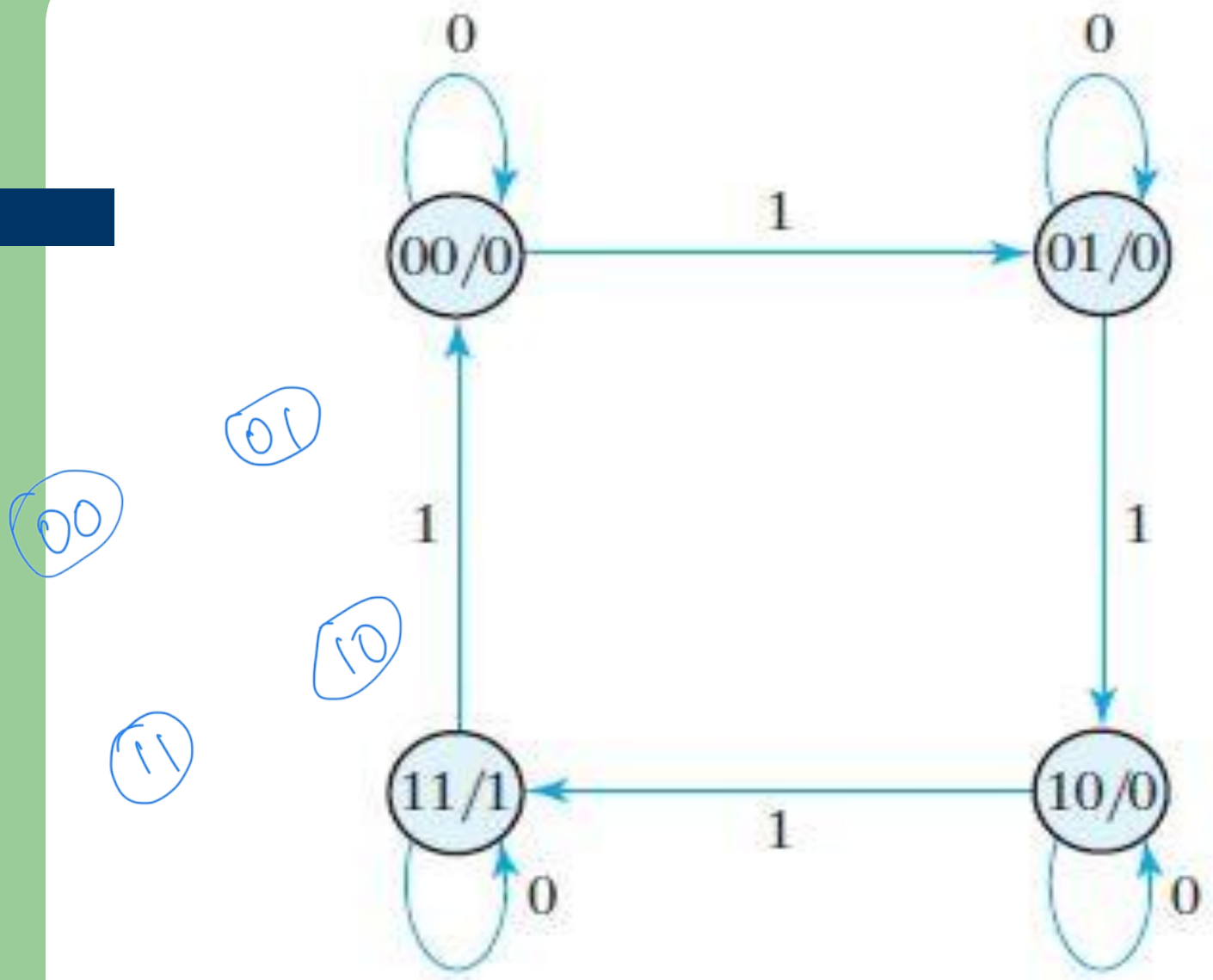
It is the Moore FSM !

- How ?

It is the Moore FSM because ..

- The outputs depend only on the flip-flop values.
- That is, the output is a function of the present state only.
- The state diagram for this is shown in the next slide.

State Diagram



Explanation

- The state diagram in this case has the input values labelled along the directed arcs.
- The output is indicated inside the circle together with the present state.

Remarks

- In a Moore model, the outputs of the sequential circuit are synchronized with the clock (since they depend only on the flip-flop outputs).
- On the other hand, in a Mealy model, the outputs may change if the inputs change during a clock cycle.

Moving on to design ..

- We have so far concentrated on analysis of sequential circuits.
- The tools we have described (state tables, state diagrams etc.) are also useful to design a circuit given some specifications.
- One may even start from an algorithm for some problem and see how the hardware will be.

Deciding the blocks for the circuit

- Different procedures for hardware design are possible.
- In general, it is useful to first simplify a design by reducing the number of gates and flip-flops as much as possible.
- Reduction in flip-flops, for example, automatically leads to a reduction in the cost of the circuit.
- Reduction in flip-flops is called as state reduction.
- We will consider an example now starting from the state diagram. Why should the state diagram have extra states at all ? Question for students.

Extra states ?

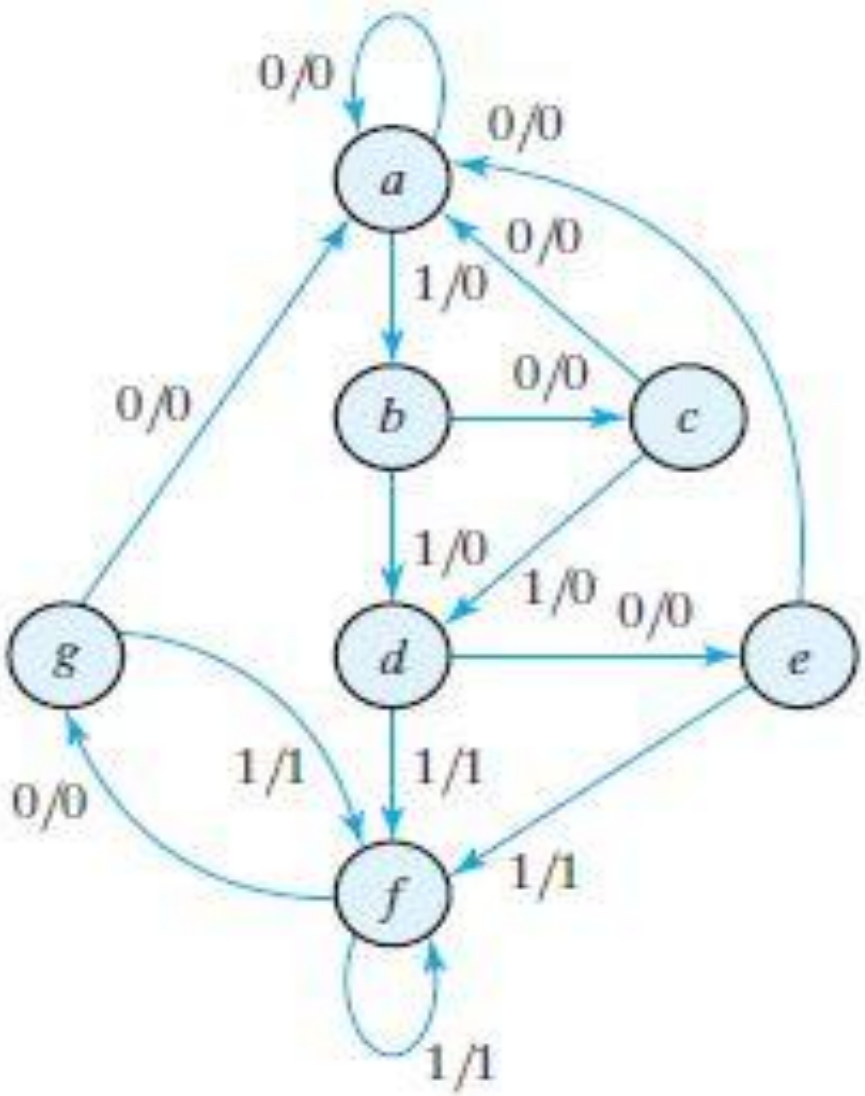
- Why should the state diagram have extra states at all ? Question for students.

More on extra states ..

- While designing control units for contemporary microprocessors, we start from an algorithm typically for computing a quantity.
- Starting from an HDL specification of the algorithm, we can identify how many states we will have (while we design the hardware) and then prepare a state diagram.
- It is possible that we have not captured the number of states accurately while designing the HDL code (or pseudocode) or while constructing the state diagram.

HDL + Hardware description language

Consider this state diagram



How to reduce states ?

- First of all, we do not need to put binary numbers inside the circles since only input-output sequences are important now. So we just label the states from a to f.
- Given the state diagram (or state graph), we need to find out which states are equivalent. If two states are equivalent, one of them can be removed without affecting the input-output relationship.
- Two states are equivalent if, for each member of the set of inputs, they give exactly the same output and send the circuit to the same state.

Extracting equivalent states ..

- In general, it is hard to get the information on equivalent states from the state diagram.
- It would be useful to have a state table prepared from the state diagram. Then one can scan the state table systematically and extract the information regarding equivalent states.
- So we show the state table next (obtained from the state diagram).

State Table

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	g	f	0	1
g	a	f	0	1

Observations

- We want to extract the equivalent states.
- Start scanning the state table from the top row and see which of the present states have the same next state and output for input value of x = 0
- We have the picture shown next.

For input of x =0, we have

Present state	Next state x=0 (input)	Output x=0 (input)
a	a	0
c	a	0
e	a	0
g	a	0

Are all the 4 states equivalent ?

- From the state table (extract) shown in the previous slide, we need to proceed to the next input, namely x = 1.
- See if the next state and output are the same.
- If they are, then all four are equivalent.

Bringing in input of x = 1, we have

	Present state	Next state x=0	Next state x=1	output x=0	output x=1 (inputs)
X	a	a	b	0	0
X	c	a	d	0	0
✓	e	a	f	0	1
✓	g	a	f	0	1

Conclusion

- So we see that only states e and g are equivalent.
- We can remove g and modify the table.
- Modification involves also replacing the next state from g to e in other rows of the table (for example, the row with f as the present state)

So the reduced table becomes

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	e	f	0	1

Can we reduce the no. of states further ?

- Once again, we need to scan the reduced table we have obtained to determine this.
- Doing so reveals that d and f are equivalent.
- So we will have the state table shown next.

So the table becomes

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	d	0	1
e	a	d	0	1

Any further reductions possible ?

- We can scan the new state table to see if the number of states can be further brought down.
- It turns out that no further optimization is possible here.

Textbook and Reference

- M. Mano and M. Ciletti, Digital Design, Pearson Education, 2018
- S. Brown and Z. Vranesic, Fundamentals of digital logic with VHDL design, McGraw Hill, 2017.